

@April 8, 2026 4:02 PM

Summary

Action Items & Next Steps

- ☐ Discard the current complex architecture and create a simplified design ▶ ▶
- ☐ Create an end-user level UI mockup showing how the interface will look ▶ ▶
- ☐ Redesign the system to query existing bug tracking systems (Jira, Bugzilla, GitHub) rather than replicating their data ▶ ▶ ▶
- ☐ Remove the unified bug submission portal concept - users should continue using their existing systems ▶ ▶
- ☐ Study modern agentic AI technologies including agent [skills.md](#) and MCP (Model Context Protocol) ▶
- ☐ Create a high-level design showing how to gather information from multiple data sources using agentic AI ▶
- ☐ Document design goals and requirements based on the revised understanding ▶
- ☐ Share updated design in chat for feedback when ready ▶

Problem Statement Presented

- HPE deals with complex hybrid cloud systems including storage, compute, networking, and software ▶
- Different engineering teams use different bug tracking systems: Jira, Bugzilla, and GitHub ▶
- A single bug can involve multiple components across different systems ▶
- Key pain points identified: ▶
 - Data is scattered across disconnected systems
 - No unified view for customers and executives
 - Lineage is lost between related issues

- The entire process is fragmented, manual, and disconnected

Proposed Solution (Initial Design)

- Agentic multi-repository bug management system acting as a single unified interface ▶
- Core concept: One bug submission triggers automatic actions across all relevant systems ▶
- System capabilities: ▶ ▶
 - Multi-repository ticket creation based on affected components
 - Consolidated updates from different systems in one view
 - Federated search allowing engineers to access related information from a single place
 - Automated classification and intelligent routing
 - Complete lineage maintenance linking all tickets back to the original issue

Architecture Components Presented

- Customer-facing UI with bug submission form ▶
- JWT authentication for user authorization ▶
- Fast API gateway (Python web server) handling front-end and back-end requests ▶
- Kafka topic for queuing and decoupling: ▶ ▶
 - Maintains queue of incoming bugs
 - Prevents data loss if orchestrator or agents are down
 - Allows agents to process bugs at their own pace without being overwhelmed
- Orchestrator coordinating agents and ensuring system can resume after failures ▶
- Agent Pipeline with four sequential agents: ▶ ▶ ▶
 - **Intake Agent:** Cleans and normalizes input, generates unique case ID

- **Classification Agent:** Uses LLM to determine severity and identify affected components, with keyword-based fallback
- **Routing Agent:** Uses database lookups to determine which teams and platforms need involvement
- **CRUD Agent:** Creates actual tickets in destination systems, checks for duplicates to ensure idempotency
- PostgreSQL database with three tables: ▶ ▶ ▶ ▶ ▶
 - **Work Cases Table:** Tracks every pipeline step, enables system to resume after crashes
 - **CMDB Team Registry Table:** Maps components to teams and their tracking systems
 - **Work Items Table:** Prevents duplicate ticket creation and powers unified view
- Connector registry translating commands to specific platform connectors ▶
- Docker Compose for containerization and deployment ▶

Critical Feedback Received

- **Major Issue - Data Duplication:** Creating a parallel data system defeats the purpose of existing systems and is impractical ▶ ▶ ▶
 - Would require duplication of efforts and data storage costs (potentially doubling infrastructure costs) ▶
 - Not possible to replicate all data due to RBAC (Role-Based Access Control) and data sovereignty concerns ▶ ▶
 - Systems handling sensitive data must remain intact ▶
- **Incorrect Assumption - New Bug Portal:** The requirement is not to create a new unified bug submission interface ▶ ▶ ▶
 - Users should continue using their existing bug tracking systems ▶
 - The goal is to query and aggregate data, not replace existing workflows ▶ ▶

- **Revised Goal Clarification:** The system should: ▶ ▶ ▶
 - Query existing bug tracking systems without duplicating their data
 - Use modern agentic AI to gather information from discrete data sources
 - Present information in a unified, meaningful manner
 - Simplify and speed up bug triage by intelligently correlating information
 - Reduce turnaround time for bug triage process
- **Missing Component - UI Mockup:** Need to show end-user perspective to clarify requirements and identify bottlenecks ▶ ▶
 - Should show how related issues, customer case notes, user stories/epics, and bugs are displayed together ▶

Design Principles to Retain

- **Good Decoupling Approach:** The team's thinking about decoupling components is sound ▶ ▶
 - Kafka-based queuing to prevent system overload is a good pattern ▶
 - Design allows parts of the system to continue functioning even if other parts break ▶
- **Modular Architecture:** The decomposition and modularization approach is valuable ▶
- **Unified View Concept:** Creating a single interface where engineers can see information from multiple bug tracking platforms without navigating to each system separately is aligned with requirements ▶ ▶ ▶
 - The idea is to show information and reference original bugs in their respective systems ▶

Meeting Logistics

- Feedback can be shared in chat between monthly meetings ▶
- Additional meeting can be scheduled if design calls for more discussion ▶
- Team should iterate on design and share when ready for another review ▶

Notes

Transcript

Yeah, so... Yeah, so let us start with our understanding of the problem statement. and the core issues we are trying to address basically in a large enterprises like hp we are dealing with a complex hybrid cloud system which include storage compute networking and software now the thing is each of these areas is handled by our different engineering teams and each team uses its own bug tracking system like zira bugzilla and github so the real issues start when a bug is reported a single bug can involve multiple components like storage, compute or networking, sometimes even all together.

But these are tracked in completely different and disconnected system. So because of this, we are some key pain points like data is a skater. It means we have to open multiple systems just to understand the issue. There is no unified view where customer and executive see the complete status in one place. And last, the lineage lost. So overall, the problem is that the entire process is fragmented, manual and disconnected.

we are addressing. Now we will move ahead and discuss the solution for this problem. Oh.

So So let me walk you through the proposed solution as Om discussed about the what all problems we were discussing. So what we are proposing is an agentic multi-repository bug management system. which will act as a single unified interface for all the bug related workloads. Our core idea is very simple. One bug submission will trigger everything automatically. The system will use agentic AIs to analyze the bug, identify the affected components, create tickets in the relevant platforms such as Jira or Bugzilla.

and at the same time it will also provide a single unified view where the entire cycle of the issue can be tread. Behind the scene the system handles multiple operations simultaneously. Firstly, it performs multi-repository ticket creation based on the affected components. Second, it consolidates updates from different systems in one unified view. Then next term is it also enables us for the federated search allowing engineers to access all the related information from a single place.

It also performs automated classification and intelligent routing which ensures that the which ensures read the correct teams without the manual intervention.

Additionally, it maintains complete lineage so every ticket remains linked back to the original customer issue. So all of this works together as one continuous flow rather than separate disconnected steps. So as a result the engineer can focus on issues relevant to them only while the system will take care of the coordination, tracking or any disturbance in the background. So overall this is transformer bug handling.

make it single intelligent automated pipeline making the entire process more streamlined consistent and efficient Okay, so let's now discuss the POC architecture that we designed.

So here we can see that the diagrammatic systematic views of various components. So it starts with a customer when he logs a bug through the bug submission form in his UI or dashboard. The bug gets reported through our UI. On the UI, there are various other details also that are present for internal bug management. After that, the bug travels through JWT authentication that ensures that the user that is reporting a bug is correctly authorized.

After the bug is authenticated and found to be genuine, the Bug is routed through a fast API gateway. It is a lightweight Python web server that handles all our front-end and back-end requests. After that, the bug goes to a Kafka topic, that is raw bugs. What Kafka does is it helps us to maintain a queue. based on all the bugs that are coming in. It also helps us to decouple the architecture. That means if a large number of bugs are incoming like in the thousands, the orchestrator or the agents don't get too overcrowded. They can take on bugs as they are ready.

Kafka stores them in memory until they are ready to be read by the orchestrator or the agent. After the bugs are aligned in the Kafka topics, they are read by the orchestrator. What the orchestrator does is it coordinates the agents that we have set up and routes the bugs through them. It also ensures that if one of the agent or a component is failing, when the system restarts, it can pick up right where it left off.

Now we move to our agent pipeline which processes the data in a strict sequence. The first agent is the intake agent. This agent takes the raw customer input, cleans it, normalizes the input text, and generates a unique case ID which it stores in the work case table in the Postgres database. The next agent is the classification

agent. So it is the intelligent core. It takes the clean text given by the intake agent and sends it to the LLM.

The LLM determines the severity of the bug and identify what components are being affected. It then updates the Postgres work system. If the LLM fails or returns invalid output, we fall back to normal keyword-based classification so the pipeline never breaks. The third agent is the routing agent, which takes the components identified by the classification agent and determines exactly which teams and bug tracking platforms need to be involved.

It does this by querying the CMDB team registry table, which is present in our course database. So the routing agent only use database lookups, no LLM calling nothing. The fourth agent is the CRUD agent, which is the actual executor. It takes the routing instruction and prepares to create the actual tickets in the destination system using these connectors. So before creating a ticket, we need to check if it already exists to avoid duplicates and ensure our system is idempotent.

Now, none of these agents give data in memory to make our agents stateless and horizontally scalable. Everything they do interacts directly with the post which stores the state. So it has a work cases table which track Every single step of the pipeline, the orchestrator keeps updating the stable. If the system crashes with processing Postgres, tell exactly where to resume from the next iteration. The next tableOh.

So, next table, any questions? Yeah. What is the role of this PostgreSQL DB that you have brought in here? To manage the state, like we are not storing any state in the agent. All state is being stored in these tables. So whenever agents need to...

When you say state, state of what?

State of what all the current bug is tracking, like the case ID, summaries, severity, status, what all components are being affected, all these are stored in the team. So whenever an agent starts...

If I understand, let's say now...

the bugzilla or jira or github issues whatever that you have let's say now i have 100 issues on each of these systems which will make it like 300 issues then will this postgres sql tv will have 300 box entries into this. Yes, all the 300 bug entries will be reviewed. then this particular table. Okay, that's not a good model to have.

Okay, go ahead. Okay, uh...

Can you enable chat also in the meeting?

It says chat is turned off, so. Mm. Please continue, but somebody, one of you please enable the chat in the middle. Okay sir, okay sir, I'm using that. Oh, so that- The next table which we use is CMDB team registry table, which we use for dynamic routing. So the routing doesn't have hard-coded logic. It queries the specific table to map a component to a specific theme and their tracker. So the next table in Postgres is work items table, which we use for idempotency and a unified view.

So before the CRUD agent creates a ticket, it takes this table outbox to ensure we haven't already created it. which prevents duplicates. Once a ticket has been created, this table stores the external URLs, which is exactly what powers the unified URL. So when the UI shows one case with ticket, one case with tickets and two trackers, this is the table it's reading from by simply using a join query.

Once the credit agent knows what tickets need to be created based on the database, it hands the execution over to our connector registry. The connector registry acts as the translator routing that command to a specific Voxela, Jira, or GitHub connector. Finally, to make this entire proof of concept reliable and easy to demo, we need a consistent way to run it. So we containerize our application and use Docker Compose to spin up the entire containers.

Yes, my request Yeah. Can you explain, there is a data source from where you will start, the flow will start where there is a particular bug ID, it starts from there. and then you will look at where are the other data sources from where you can gather the information about that particular issue. So we are talking about a bug here, right? So in that, from that workflow context, can you map it and Explain from that perspective, from a workflow context and also why is Kafka required and what are you storing in the database?

So it is like, are you trying to persist whatever queries user is making? Like what exactly is the intent behind it? This is our design team. From end-user perspective in terms of the workflow, and try to correlate that with the design of the architecture that you are explaining here. That will be helpful. Okay. Shivansh, can you take that flow that we were discussing from end to end? Yeah, sure. So let's say a customer is having some issues on the payment gateway like Stripe API is returning some error, it's not processing.

So when he goes to log in the bug in a dashboard, so this is the primary entry point for a bug. Like he says that I'm not able to complete my payment or the Stripe web page is returning 504 error or something like that. So after the bug is like in our system, we push it through the entry layer. That means it has assigned a case ID and it is logged into our UI. It is shown there. What is the status of the bug? How much time it has to...

How much time it has been since it has been logged, etc. Then the flow continues like the bug is authenticated, the fast API handles the the request and pushes it to Kafka. Coming to your question why we are using Kafka is sometimes like the customer logs in the bug but our orchestrator is down or some agent is not working. or something like that. So we don't lose the bug in the process. That's why we are storing it in a Kafka topic.

Yeah. Okay. So my thing is like, see, it is like already there is an enterprise, right? We have projects, multiple projects, and each project has several set of components. And based on those component categories, there are several issues which are already there. So ideally, your data source are those reports or list of bugs that are there. Out of that, for one particular bug, let's say, if I give an input into your agentic bug, tool which is probably a customer found issue or it could be a like you know product issue or a it could be a story or whatever that particular issue id once i like enter in in your system it's supposed to gather let's say if it's a customer issue there will be a different portal where those like whatever observations that were made in the case nodes all of that information and in the actual There could be a user story around that based on which the feature was invented.

So that detail and in Bugzilla maybe the issue that was found. So that details all of that. You are supposed to fetch it. So I think like why submission? What is submission form and why are you talking about new bug creation and all that? Like so this was the POC simplified version that we decided to show like we also had a full architecture that was scraping bugs from all these input sources. So I think you are talking about this like developer also can report the bug. There are some context on the GitHub page. Some some are there on the Jira tickets. So we are fetching all of that scraping basically and then putting it.

The bugs list of bugs.

You have a data source where you get the list of bugs when you authenticate. having access to a certain like HP product and you have access to all of the bugs and you should be given a API token you should be able to query and get the bug details so assume that and start from there so if you want to make changes according to that you can do it and in the next iteration you can show that yeah so keeping that aside you can continue like what was the underlying design that you wanted to like Yeah, yeah, sir.

Yes, Pulkit, you can go on. Yes. Cool. Like we are handling two issues here from our understanding of the problem statement. One is if a new bug is being reported that we handle to our triage system. The second is the existing bug sources which are already there, Zira, Bugzilla, GitHub issues, all the bug issues that are present. So we are also using them as a, we are scraping that data, putting it into our pipeline and identifying those components.

And we are storing all of that in our database also so that we could fetch from our unified everything. We are addressing these two issues. Are we addressing only the second issue that you talked or we are not taking new bugs in hand? they will not have access to other systems they will have access to that particular portal from where they will create the issue so those the discrete or different data sources are going to be there so there is not going to be a one unified portal from where everybody can log the issue so that we nowhere mentioned that as a requirement so you can take it out rajesh and rajesh divakar and barati can like call out if there is a I mean gap year So we don't want such a system where we want to-Yeah, yeah.

When we are bringing in that single source, that data plane that you're showing with the Postgres database, that is like creating a parallel data system for composing of all the systems that what we have. It really defeats the purpose of that end systems what are there. And also, practically, that is not possible to have that data replicated in some place and then and also from business wise that doesn't make sense.

Yeah, so we don't want to create a huge repository which combines all the issues across multiple systems. All you are the ask is that whatever existing systems are there, you should design a system where you will be able to query the query and use utilize the modern agentic AI skills. You can use skills.md right if you are

aware of that. So try to utilize the modern agentic AI system or components to gather all the data sources.

whatever existing systems are there, you should be able to query them. Then you create a platform where you can show in a unified manner. show the information in a meaningful and unified manner. So as part of this, maybe you could also come up with a mock-up in terms of how the end, like the UI will look like where all of these related issues like there could be a notes, case notes from customer and there could be original from where the requirement originated. So the user stories or the epic and then the actual bug that is being trashed.

Then if there is an issue, aspects that you can tie in which can help in terms of doing the bug triage. So we want to say we want our main goal is to simplify the bug triage in an agentic way to gather all the necessary information which is available in discrete data sources gather them and then intelligently you know make meaningful sense out of it to simplify or reduce the turnaround time to do the to perform the bug triage.

So that is the ask here. See whatever design that you have come up with is one of the possible options that one can take. So you are basically taking the data from all the sources and putting it in one place and then you are trying to decipher that. So that is a natural thing that, OK, if I have everything in one place, then I can derive decisions out of that. That is definitely one of the solutions that you can have.

Practically, that will not be possible and also it is impractical to get all those systems data into one place and then delivering it. when that will require duplication of efforts and duplication of the data and and also we You need to...

Since that whatever data that is available in the individual systems, right, those are all RBAC controlled, right? We cannot say that, okay, I'm going to get every data in one place from a data sovereignty perspective also. We cannot do that. Yes, sir. Okay. Okay. the agentic way of gathering all the required information. Then there will be an orchestrator engine which will make sense out of all of that discrete information. Based on the incoming bug that needs to be triaged, then based on that you need to have workflow in your system which can help speed up the triage or give more meaningful information to the user.

and to the development development team or whatever is the user. Oh, okay. Got it. Yeah. So go ahead and make notes of all these inputs and you can incorporate

that. And also most importantly, come up with an end-user level mockup showing how the end picture will look like in terms of the UI. That will give you some good insight in terms of how you want to go back and design the architecture. And where the bottlenecks can be and all of that will become clear to you.

And with things to avoid will also become much more clear. Okay, okay.

Okay. Okay, sir.

okay so like have you done the design for uh you know like uh the things that i talked about where uh we are connecting to connector registry i saw that but uh yeah like have you done uh study i think you have presented already right like what are the agent technologies that you are going to planning to use and all that uh so sorry i forgot but uh Consider the agent skills or MCP or things like that and then see if you can give a high level design in your next iteration when you are presenting in terms of how you are collating the are gathering the information from multiple data sources.

Chance. Okay. Yes. Is there any other diagram other than this? Yes, sir, we sent you two diagrams. This one is a more detailed.

Fuller archetype.

Detailed one, but again, it has all those assumptions that you're talking about. So clean all that up. This is like... Very complex, honestly, the diagram that is there. That's the reason the moment I saw I said you have to simplify. Discard this, try to simplify. and also keep the end goal in mind. So to start with, first come up with a sort of a design goals and also from a requirement perspective, we have already listed there, come up with a mockup based on which then you can then back trace and go back and like revisit this simplified design that you want to work on and then you can share it again.

Okay.

Uh...

Sir Yeah. Can we increase the frequency of meetings so that we could have more iterations on the architect? Yeah, you can share. So there are inputs and if it calls for a discussion, definitely we can meet. So we prefer to meet once a month only. But if you need something additional, you can feel free to message that in the chat and then we'll try to answer as much as possible. So having this in-person meeting, we want to keep it once a month ideally.

But if it calls for another discussion, maybe one more time we can do that. I need this. Okay. Sorry, this is Bharat. I was just listening to the question. I just want to make sure, was the problem description not clear? What made you come to the conclusion that the ask is to create a problem? whole a bug tracking system in itself. Just want to make sure you know, that I understand what aspect of the description was not clear, number one. Some of these components, you will need it there as well, right?

When you define that the workflow, the agent-KI system for that. You don't necessarily have to throw all of it away. Remove everything that you have put together to actually build that an equivalent of a terra think about it right so replace that with maybe call it a source endpoint right so because as one of some of us mentioned you know we have multiple systems some of us back box in github some people use terra some people use something else. So the source of it has been really important.

It is that your system should be able to sort of use the tool, the information that comes from the tool, either through some kind of web socket that gets established or something like a data could expose a Kafka topic that one of your components can listen on for any bugs that get created there. So again, you don't have to throw I like the fact that you guys It's part of decoupling the system where if one thing breaks, the other one, the entire system sort of still remains.

From a principle point of view, these are good design thinking. It's just that the components that they connect to aren't necessary. So the question that I had, first one was, what made you guys think that you had to build an entirely full-blown bug tracking system? And second, what is the AI-specific components that you've put together that is outside of the ...system itself.

I thought somebody had their hand up.

So how we were thinking of it is not replicating the entire bug system. We were considering like we have underlying bug systems, we have to make a unified UI out of them so that we need not to go to multiple bug systems to think of the same logic. That was not the ask, yeah. Yeah, yeah. Sorry, I have a hot stop at 4.30, so I'll be dropping off. Thanks, everyone. Yeah. Good evening, Pete. Yes, so how we were approaching it was to build a unified web over all the bug tracking platforms so that an engineer need not to go to multiple systems to see about a single bug

because a single bug can be reported in two bug platforms if two components are affected for two different teams.

One has to monitor both Jira and Bugzilla for the same thing. So what we approached was so that the engineer only sees a single platform where he could fetch data from both the bug reporting platforms and see all the results there on a single platform. That's how we approached it. Okay. I mean, I kind of like that idea, Jivakar, Ajay, as opposed to, I don't know if it came out as a completely different interface for creating issues.

So I remember in one of the discussions what we were told was that you got to sort of link. When I say link, I think you show your information about the issue, but then you reference the original bugs in those two systems. That was one of the things that we had asked him, right? In that sense, These guys are on on the same track that we had.

Right, ready to work out? Yeah, yeah, that's right. Yeah.

Yeah, cool. Okay, I think that sort of clarifies. You're sort of on the right track, but the way it came out, or at least the way the design is coming out is that you're sort of trying to replace or add an overlay, an app action or an overlay on top of all the systems, even for creating bugs. And that I think will not fly. That would essentially mean that we are asking, we're gonna ask everyone to sort of Peace.

with a new system and we don't want to do that. Maybe sometime in the future it could get too complex underneath but at this point it's not so I think it's okay. So yeah, I think you've got the input that you know that from us we like some of the things will be like the way you thought of you know, decoupling and all of that. I think that's very... Yeah. I don't want you to think of it that way. But yeah, take the...

comments that you received and you know, and then yeah, we can catch up whenever you guys are ready. for another review Oh, okay. Definitely this is a very good learning for you guys that okay you thought of this design and then now you are getting to know that okay what is not what will not work with this design. And also what you need to get out of this effort what you have done here is that there is no point in duplicating the data. And also from a legality perspective also we may not be able to duplicate this data or pull out this data into a single source and there are a lot of data sovereignty things that would play in here.

So that's where we need to have the systems which handles this data, those things needs to be intact and those data should go into only those systems. We cannot duplicate this data here and also If you see from a technical perspective, It doesn't make sense for us to say if I'm spending say petabytes of data for these three or four systems that I have and if I need the same number of petabytes of data again to create that overlay on top of these things. So I'm really incurring double the cost of the systems or what I already have.

Yes sir. Oh, okay. Thank you.

Got it.

Definitely, I appreciate the thought process what you guys are having. It's only that now you need to think about whatever that you have thought about decomposing and Making it as modularized, that's a good thing. Sure, sir. We'll reiterate.

Yep, yep. Okay That's all we have to discuss. Right thanks. Okay. Thank you, sir.

Thank you so much. Thank you, sir.

We are leaving.

Make a new meet, we have 12 minutes left.

Right. This is the name of the temple.

Good. I woke up. - Ooh.